Vehicle Parking App - V2

STUDENT NAME: Rohit Rai

STUDENT EMAIL ID: 23f1000362@ds.study.iitm.ac.in

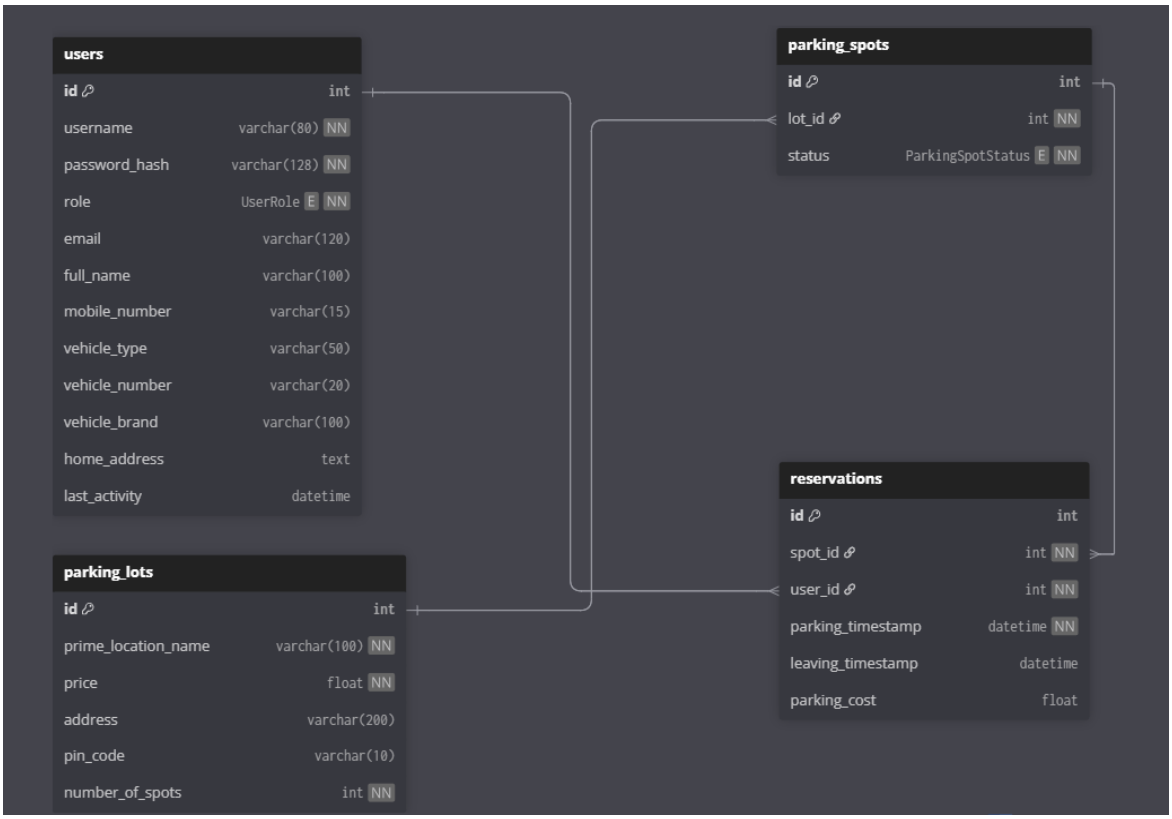
DESCRIPTION

The Vehicle Parking App is a comprehensive multi-user web-based platform built using modern technologies that manages parking lots, parking spots, and vehicle reservations for 4-wheeler parking. The application is designed with two distinct roles: Admin (superuser) and Users (customers). The admin has complete control over parking lot management and system oversight, while users can register, browse available parking lots, reserve spots, and track their parking history. The platform offers advanced features including real-time spot allocation, automated cost calculation, performance analytics, scheduled notifications, and comprehensive reporting—all delivered through a responsive and intuitive interface powered by VueJS and Flask.

TOOLS USED

- Flask: A lightweight Python framework used for building the robust backend API architecture of the application.
- VueJS: A progressive JavaScript framework utilized for creating dynamic and responsive user interfaces.
- Jinja2: A templating engine employed for the entry point when using CDN-based implementations.
- SQLite: A lightweight relational database system used for efficient local data storage and management.
- Bootstrap: Integrated for responsive design and visually appealing front-end styling across all devices.
- Redis: Implemented for high-performance caching and as a message broker for background task processing.
- Celery: Used for handling asynchronous background jobs, scheduled tasks, and batch processing operations.
- Chart.js: Integrated to create interactive visualizations for parking analytics and performance tracking.

DATABASE SCHEMA DESIGN



The database schema is designed to efficiently manage user accounts, parking infrastructure, and reservation workflows using a relational structure with clear relationships to ensure data integrity and real-time availability tracking.
USER Table: Manages user credentials, profile details, and role information of admin and user.
PARKING_LOTS Table: Stores parking lot metadata including location, pricing, and capacity.
PARKING_SPOTS Table: Tracks individual parking spot status within each lot.
RESERVATIONS Table: Records each parking event with timestamps and cost calculation.

This schema ensures seamless coordination between users, parking resources, and reservation history.

SYSTEM FUNCTIONALITY

Authentication and Role Management

- Admin login: Predefined superuser with root privileges (no registration).
- User registration/login: Email, password, full name, and contact details.
- JWT-based access control: Role-specific access to admin and user dashboards.

Admin Dashboard

- Parking lot management: Create/edit/delete lots (delete only if empty).
- Spot auto-generation: Bulk creation of spots per lot capacity.
- Real-time status view: Monitor all spots availability and occupied details.
- User overview: List all registered users and their parking histories.
- Analytics charts: Visual summaries of lot usage and revenue (Chart.js).

User Dashboard

- Lot selection: Browse available lots and pricing.
- Automatic allocation: First-available spot assignment.
- Parking operations: Check-in/check-out with timestamped status updates.
- Cost calculation: Auto-compute cost based on duration and lot pricing.
- History and analytics: View past reservations and personalized usage charts.

BACKEND JOBS (Celery + Redis)

- Daily Reminders: Evening alerts via Google Chat webhook/SMS/email for inactive users.
- Monthly Reports: HTML activity summaries emailed on the 1st of each month.
- CSV Export: User-initiated batch job exporting reservation details and cost breakdown.

PERFORMANCE AND CACHING

- Redis caching: Frequently accessed endpoints (available lots/spots) with expiry policies.
- API optimization: Cache refresh strategies to minimize database load.

IMPLEMENTATION HIGHLIGHTS

- Modular Architecture: Flask Blueprints separating auth, admin, user, and API controllers.
- VueJS Components: Reusable UI modules for dashboards, forms, and charts.
- Programmatic Schema Creation: SQLAlchemy models ensure automated table creation.
- Asynchronous Processing: Celery workers and Redis beats handle scheduled and user-triggered jobs.
- Secure Authentication: JWT tokens with route protection decorators.
- Responsive Design: Bootstrap-powered UI optimized for desktop and mobile.

VIDEO LINK: https://drive.google.com/file/d/1i-mW3WTk9snfR5NuCYlDNMAt555j4yOM/view?usp=sharing

This Vehicle Parking App V2 delivers a robust, scalable solution for urban parking management, combining real-time operations, advanced analytics, and automated background processing in a unified, user-friendly platform.